

บทที่ 4 การแสดงผลข้อมูล

วัตถุประสงค์การเรียนรู้

1. อธิบายความหมาย เป้าหมาย และแรงจูงใจของการแสดงผลข้อมูล รวมถึงแนวคิดหลัก ได้แก่ Representation, Arrangement และ Selection
2. จำแนกประเภทเทคนิคการแสดงผลข้อมูลตามจำนวน attributes และลักษณะข้อมูล และระบุหลักเกณฑ์การเลือกกราฟที่เหมาะสมกับข้อมูลแต่ละประเภท
3. อธิบายทฤษฎีและจุดเด่นของกราฟพื้นฐาน ได้แก่ Histogram, Box Plot, Scatter Plot, Bar Chart, Pie Chart และ Line Plot รวมถึงข้อดีและข้อจำกัดของแต่ละประเภท
4. สร้างกราฟประเภทต่างๆ ด้วย Pandas ได้แก่ Scatter, Line, Area, Bar, Histogram, Box, Pie และ Color Map ด้วยฟังก์ชัน `df.plot()`
5. สร้างและปรับแต่งกราฟด้วย Matplotlib ได้แก่ การเปลี่ยน Marker, สี, รูปแบบเส้น, ขนาด, Colorbar, Violin Plot และการสร้าง Multi-Plots แบบ Subplot
6. สร้าง Relational Plots ด้วย Seaborn โดยใช้ `sns.relplot()` รองรับการแบ่งกลุ่มด้วย hue, col และ size ทั้งแบบ Scatter และ Line
7. สร้าง Distribution Plots ด้วย `sns.displot()` รองรับ Histogram, KDE, ECDF และ Rug Plot และสร้าง Categorical Plots ด้วย `sns.catplot()` รองรับ Strip, Box, Violin และ Bar Plot
8. สร้าง Joint Plots ด้วย `sns.jointplot()` และ Pair Plots ด้วย `sns.pairplot()` เพื่อแสดงความสัมพันธ์ระหว่างตัวแปรหลายคู่พร้อมกัน
9. ประยุกต์ใช้การแสดงผลข้อมูลกับชุดข้อมูลจริง ได้แก่ ข้อมูลเศรษฐกิจสหรัฐอเมริก และข้อมูลทิปร้านอาหาร (tips dataset)

4.1 บทนำ

การแสดงผลข้อมูล (Data Visualization) คือกระบวนการสร้างการแสดงผลข้อมูลในรูปแบบกราฟิกเพื่อสื่อสารข้อมูลและข้อมูลเชิงลึกได้อย่างมีประสิทธิภาพ เป้าหมายคือการใช้องค์ประกอบภาพ เช่น แผนภูมิ กราฟ และแผนที่ เพื่อทำให้ข้อมูลเข้าถึงได้ง่าย เข้าใจง่าย และนำไปปฏิบัติได้สำหรับกลุ่มเป้าหมายที่แตกต่างกัน

Python มีแพ็คเกจที่ใช้กันทั่วไปสำหรับการแสดงผลข้อมูล ได้แก่:

10. Pandas: บูรณาการกับไลบรารีอื่นๆ เช่น Matplotlib และ Seaborn เพื่อสร้างกราฟและการแสดงผลประเภทต่างๆ ที่ช่วยให้เข้าใจข้อมูลและระบุรูปแบบและแนวโน้มได้
11. Matplotlib: ไลบรารีสร้างกราฟ 2D ที่มีเครื่องมือหลากหลายสำหรับการแสดงผลแบบ Static, Animated และ Interactive ใช้เป็นรากฐานสำหรับไลบรารีอื่นๆ อย่างแพร่หลาย
12. Seaborn: ไลบรารีที่สร้างบน Matplotlib มีอินเทอร์เฟซระดับสูงสำหรับสร้างกราฟทางสถิติที่สวยงามและให้ข้อมูลเชิงลึกมากขึ้น เหมาะสำหรับการแสดงผลข้อมูลในสถิติและวิทยาศาสตร์ข้อมูล
13. Plotly: ไลบรารีสำหรับการแสดงผลแบบ Interactive และบนเว็บ มีเครื่องมือหลากหลายสำหรับสร้างกราฟ แผนที่ และ Dashboard เหมาะสำหรับการฝังการแสดงผล Interactive ในหน้าเว็บหรือแอปพลิเคชัน
14. PyViz: ไลบรารีที่ประกอบด้วยชุดไลบรารี เช่น Holoviews, Geoviews, Datashader และอื่นๆ สำหรับสร้างการแสดงผลสำหรับข้อมูลที่ซับซ้อนและชุดข้อมูลขนาดใหญ่

4.2 แนวคิดและทฤษฎีที่เกี่ยวข้อง

4.2.1 ความหมายและเป้าหมายของการแสดงผลข้อมูล

การแสดงผลข้อมูล (Data Visualization) คือการแปลงข้อมูลให้อยู่ในรูปแบบกราฟิกหรือตารางเพื่อให้ผู้ใช้สามารถวิเคราะห์และรายงานข้อมูลเชิงลึกได้อย่างมีประสิทธิภาพ Tan, Steinbach และ Kumar (2006) นิยามว่าเป้าหมายหลักของการแสดงผลข้อมูลคือการตีความสารสนเทศจากภาพโดยมนุษย์และการสร้างโมเดลทางความคิดของข้อมูลนั้น ซึ่งต่างจากเครื่องมือทางคณิตศาสตร์หรือสถิติที่ประมวลผลโดยอัลกอริทึม การแสดงผลข้อมูลอาศัยความสามารถของมนุษย์ในการดูดซับข้อมูลปริมาณมากจากภาพและค้นหารูปแบบได้อย่างรวดเร็ว ในบางกรณีการใช้เทคนิค Visualization ในการทำการทำเหมืองข้อมูลยังเรียกว่า Visual Data Mining

การแสดงผลข้อมูลมีแรงจูงใจหลักสองประการ ประการแรกคือมนุษย์สามารถดูดซับข้อมูลจากภาพปริมาณมากและค้นหารูปแบบในนั้นได้อย่างรวดเร็ว ตัวอย่างเช่น ภาพแผนที่อุณหภูมิผิวน้ำทะเล (SST) เพียงภาพเดียวสามารถสรุปข้อมูลจากตัวเลขราว 250,000 ค่าได้ภายในไม่กี่วินาที ประการที่สองคือการใช้ประโยชน์จากความรู้เฉพาะทาง (Domain Knowledge) ที่ฝังอยู่ในความเข้าใจของผู้เชี่ยวชาญ ซึ่งยากต่อการถ่ายทอดเป็นเครื่องมือทางสถิติหรืออัลกอริทึม การให้ผู้เชี่ยวชาญด้านสาขานั้นตรวจสอบภาพแสดงผลข้อมูลจึงเป็นวิธีที่ดีที่สุดในหลายกรณีสำหรับการค้นหารูปแบบที่มีความหมาย

4.2.2 แนวคิดทั่วไปของการแสดงผลข้อมูล

Tan et al. (2006) ได้นำเสนอแนวคิดพื้นฐานสามประการที่เป็นรากฐานของการแสดงผลข้อมูลที่มีประสิทธิภาพ ได้แก่ Representation, Arrangement และ Selection

การแทนค่า (Representation) คือกระบวนการแปลง objects, attributes และความสัมพันธ์ในข้อมูลให้เป็น graphical elements เช่น จุด เส้น รูปร่าง และสี

หลักการสำคัญคือต้องพิจารณาประเภทของ attribute ในการเลือก graphical element ที่เหมาะสม โดย attributes เชิงลำดับ (Ordinal) และต่อเนื่อง (Continuous) สามารถแทนด้วย graphical features ที่มีลำดับโดยธรรมชาติ เช่น ตำแหน่งบนแกน x, y, z ความเข้มของสี หรือขนาด ในขณะที่ attributes เชิงหมวดหมู่ (Categorical/Nominal) ควรแทนด้วย graphical features ที่ไม่มีลำดับ เช่น รูปร่างที่แตกต่างกัน หรือสีที่แตกต่างกันโดยระมัดระวังไม่ให้เกิดความเข้าใจผิดเรื่องลำดับ

การจัดเรียง (Arrangement) หมายถึงการวาง graphical elements ในพื้นที่ที่แสดงผลลัพธ์ที่เหมาะสม การจัดเรียงที่ดีสามารถเปิดเผยความสัมพันธ์ที่ซ่อนอยู่ในข้อมูลได้ทันที ตัวอย่างคลาสสิกคือการสลับแถวและคอลัมน์ในตารางข้อมูล ซึ่งอาจเปลี่ยนตารางที่ดูไม่มีรูปแบบให้เห็นกลุ่มข้อมูลที่ชัดเจนขึ้นมาทันที ในทำนองเดียวกัน กราฟเครือข่ายที่แยก Connected Components ออกจากกันจะทำให้เข้าใจโครงสร้างได้ง่ายกว่าการวาดทับซ้อนกัน

การคัดเลือก (Selection) คือการกำจัดหรือลดความสำคัญของ objects และ attributes บางส่วนเนื่องจากข้อมูลที่มี attributes จำนวนมากไม่สามารถแสดงผลได้ครบถ้วนในมุมมองเดียว วิธีแก้ไขที่ใช้กันทั่วไปคือการเลือก subset ของ attributes (มักสองตัว) สำหรับการแสดงผล หรือสร้าง Matrix of Bivariate Plots เพื่อแสดงความสัมพันธ์ทุกคู่พร้อมกัน ซึ่งก็คือหลักการของ Pair Plot ใน Seaborn นั่นเอง นอกจากนี้ยังอาจใช้เทคนิค Dimensionality Reduction เช่น PCA เพื่อลดจำนวนมิติก่อนการแสดงผล

4.2.3 การจำแนกประเภทเทคนิคการแสดงผลข้อมูล

Tan et al. (2006) จำแนกเทคนิคการแสดงผลข้อมูลตามจำนวน attributes ที่เกี่ยวข้องและลักษณะพิเศษของข้อมูลออกเป็นสามกลุ่มหลัก ได้แก่ การแสดงผล attributes จำนวนน้อย การแสดงผลข้อมูลเชิงพื้นที่และเวลา และการแสดงผลข้อมูลที่มี attributes จำนวนมาก โดยแต่ละกลุ่มมีเทคนิคที่เหมาะสมแตกต่างกัน

4.2.4 ทฤษฎีกราฟพื้นฐาน: Histogram, Box Plot และ Scatter Plot

Sturges' Rule

$$k = \lceil \log_2 n \rceil + 1$$

โดยที่:

k = จำนวนช่วงชั้น (Number of Bins) ที่เหมาะสม

n = จำนวนข้อมูลทั้งหมด

$\lceil \cdot \rceil$ = ฟังก์ชัน ceiling (ปัดขึ้นไปจำนวนเต็มที่ใกล้ที่สุด)

กฎของ Sturges ช่วยกำหนดจำนวน bin ที่เหมาะสมสำหรับ Histogram

โดยอิงจากขนาดข้อมูล เช่น $n=100$ ได้ $k=8$, $n=1000$ ได้ $k=11$

Histogram (ฮิสโตแกรม) เป็น Stem and Leaf Plot ประเภทหนึ่ง ซึ่งแสดงการกระจายของค่าของ attribute หนึ่งๆ โดยแบ่งช่วงค่าที่เป็นไปได้เป็น bins และนับจำนวน objects ที่ตกอยู่ในแต่ละ bin สำหรับข้อมูลเชิงหมวดหมู่แต่ละค่าถือเป็น bin หนึ่ง bin ส่วนสำหรับข้อมูลต่อเนื่องจะแบ่งช่วงค่าเป็น bins ที่มีความกว้างเท่ากัน (โดยทั่วไป) ข้อดีที่สำคัญของ Histogram คือสามารถเปิดเผยลักษณะของการกระจายข้อมูล เช่น ความสมมาตร ความเบ้ และการเป็น Multimodal ซึ่งยากต่อการตรวจจับด้วยสถิติตัวเลขเพียงอย่างเดียว การเลือกจำนวน bins ที่เหมาะสมมีผลสำคัญต่อการตีความ bins น้อยเกินไปจะซ่อนรายละเอียด ขณะที่ bins มากเกินไปจะทำให้เห็น noise แทนรูปแบบที่แท้จริง

Box Plot (กราฟกล่อง) เป็นวิธีแสดงการกระจายของ numerical attribute โดยใช้ข้อมูลเชิง percentile ที่กระชับและมีข้อมูลเชิงลึกสูง โครงสร้างของ Box Plot ประกอบด้วย กล่องสี่เหลี่ยมที่มีขอบล่างที่ Percentile ที่ 25 และขอบบนที่ Percentile ที่ 75 เส้นกลางกล่องแทน Percentile ที่ 50 (มัธยฐาน) เส้น “หนวด” (Whiskers) บนและล่างแสดง Percentile ที่ 10 และ 90 ตามลำดับ และจุดนอกหนดแสดง Outliers ด้วยสัญลักษณ์ “+” ข้อดีของ Box Plot คือกระชับและสามารถแสดงหลาย attributes

หรือหลายกลุ่ม ข้อมูลในแผนภาพเดียวกันได้ โดยเฉพาะมีประโยชน์ในการเปรียบเทียบการกระจายข้อมูลระหว่างกลุ่มต่างๆ

Scatter Plot (กราฟกระจาย) มีการใช้งานหลักสองประการ ประการแรกคือการแสดงความสัมพันธ์ระหว่างสอง attributes ในลักษณะ 2D หรือ 3D ซึ่งสามารถใช้ตรวจจับทั้งความสัมพันธ์เชิงเส้นและไม่เชิงเส้น ประการที่สองคือเมื่อมี class labels พร้อม จะสามารถใช้ตรวจสอบว่า attributes ทั้งสองแยกกลุ่มข้อมูลได้ดีเพียงใด หากสามารถลากเส้นแบ่งกลุ่มได้ชัดเจน แสดงว่า attributes คู่่นั้นมีศักยภาพในการจำแนกกลุ่ม (Classification) ที่ดี Scatter Plot ยังสามารถขยายให้แสดง attributes เพิ่มเติมได้โดยการใช้สี รูปร่าง และขนาดของ marker เพื่อเข้ารหัส attributes ที่สามสี่ และห้า อย่างไรก็ตาม ต้องระมัดระวังไม่ใส่มิติมากเกินไปจนทำให้แผนภาพยากต่อการตีความ

4.2.5 ทฤษฎีกราฟเพิ่มเติม: Bar Chart, Pie Chart และ Line Plot

Bar Chart (แผนภูมิแท่ง) เหมาะสมที่สุดสำหรับการเปรียบเทียบค่าของ attribute เดียวกันระหว่างกลุ่มหมวดหมู่ที่แตกต่างกัน ใช้ความสูงหรือความยาวของแท่งแทนขนาดของค่า Bar Chart มีข้อดีเหนือ Pie Chart ในงานวิชาการและเทคนิค เนื่องจากมนุษย์สามารถเปรียบเทียบความยาวของแท่งได้แม่นยำกว่าการเปรียบเทียบพื้นที่ของชิ้นส่วนวงกลม ทำให้ Bar Chart เป็นที่นิยมมากกว่าในสิ่งพิมพ์ทางวิทยาศาสตร์และเทคนิค

Pie Chart (แผนภูมิวงกลม) มีลักษณะคล้าย Histogram แต่ใช้กับ attributes เชิงหมวดหมู่ที่มีค่าไม่มากนัก แทนที่จะแสดงความถี่สัมพันธ์ด้วยพื้นที่หรือความสูงของแท่ง Pie Chart ใช้พื้นที่ส่วนของวงกลมเป็นตัวแทนความถี่ แม้ Pie Chart จะนิยมใช้ในสื่อทั่วไป แต่ Tan et al. (2006) ระบุว่าในงานวิชาการและเทคนิค Histogram มักถูกนำมาใช้แทนเนื่องจากมนุษย์ประเมินขนาดพื้นที่ส่วนวงกลมได้ยากกว่าความยาวแท่ง

Line Plot (กราฟเส้น) เป็นกราฟที่เหมาะสมที่สุดสำหรับข้อมูลที่มี temporal หรือ sequential attribute

โดยแสดงแนวโน้มและรูปแบบการเปลี่ยนแปลงของค่าตามลำดับเวลา ความต่อเนื่องของเส้นสื่อสารถึงลักษณะต่อเนื่องของข้อมูลตามเวลา ทำให้ผู้ดูสามารถรับรู้ทิศทางแนวโน้ม ความผันผวน และรูปแบบวัฏจักรได้ง่าย ในขณะที่ Empirical Cumulative Distribution Function (ECDF) เป็นรูปแบบพิเศษของ Line Plot ที่แสดงสัดส่วนสะสม (Cumulative Proportion) ของข้อมูลที่มีค่าน้อยกว่าหรือเท่ากับค่า x แต่ละค่า ซึ่งให้ข้อมูลเชิงลึกเกี่ยวกับการกระจายตัวของข้อมูลได้อย่างแม่นยำกว่า Histogram ที่ขึ้นกับการเลือกจำนวน bins

4.2.6 หลักเกณฑ์การเลือกกราฟที่เหมาะสม

การเลือกกราฟที่เหมาะสมกับข้อมูลเป็นทักษะสำคัญในการแสดงผลข้อมูล ตารางที่ 4.1 สรุปหลักเกณฑ์การเลือกกราฟตามประเภทข้อมูลและวัตถุประสงค์

ประเภทกราฟ	ประเภทข้อมูลที่เหมาะสม	วัตถุประสงค์	จำนวน Attributes
Histogram	ต่อเนื่อง / เชิงหมวดหมู่	การกระจายตัว	1
Box Plot	ต่อเนื่อง	การกระจาย / Outlier	1 (เปรียบเทียบกลุ่มได้)
Scatter Plot	ต่อเนื่อง	ความสัมพันธ์ระหว่างตัวแปร	2–3 (ขยายได้ด้วยสี/ขนาด)
Bar Chart	เชิงหมวดหมู่	เปรียบเทียบระหว่างกลุ่ม	1–2
Line Plot	ลำดับเวลา / Sequential	แนวโน้มตามเวลา	1–2
Pair Plot / Scatter Matrix	ต่อเนื่อง	ความสัมพันธ์ทุกคู่	หลาย (Selection)

ที่มา: ปรับปรุงจาก Tan, Steinbach, and Kumar (2006), *Introduction to Data Mining*, Chapter

3

4.3 การแสดงผลข้อมูลด้วย Pandas

การแสดงผลข้อมูลเป็นส่วนสำคัญของการวิเคราะห์ข้อมูล และไลบรารี Pandas ตอบสนองความต้องการนี้ ด้วยการมีฟังก์ชันในตัวสำหรับการแสดงผลข้อมูล การแสดงผลข้อมูลมีความสำคัญและจำเป็นอย่างยิ่ง ในธุรกิจปัจจุบัน Pandas จึงมีฟังก์ชัน Plot ในตัวที่ใช้งานได้สะดวก

4.3.1 แบบฝึกหัด – การแสดงผลข้อมูลด้วย Pandas

4.3.1.1 การตั้งค่า

```
import numpy as np
import pandas as pd
df = pd.read_csv('/content/Economy_of_US.csv')
```

ชุดข้อมูลนี้ประกอบด้วยข้อมูลเศรษฐกิจของสหรัฐอเมริกา (ปี 1980–2027) จำนวน 48 รายการ มีคอลัมน์หลัก ได้แก่ Year, GDP_PPP, GDP_PerCapita_PPP, GDP_Nominal, GDP_PerCapita_Nominal, GDP_Growth, Inflation, Unemployment และ Inflation_Change

4.3.1.2 Scatter Plot (กราฟกระจาย)

ใช้สำหรับแสดงความสัมพันธ์ระหว่างสองตัวแปร โดยระบุพารามิเตอร์ `x`, `y` และ `kind='scatter'`:

```
df.plot(x='Year', y='Inflation',  
kind='scatter')
```

4.3.1.3 Line Plot (กราฟเส้น)

ใช้สำหรับแสดงแนวโน้มของข้อมูลตามช่วงเวลา:

```
df.plot(y='Inflation', kind='line')  
df.plot(x='Year', y='Inflation', kind='line')
```

4.3.1.4 Area Plot (กราฟพื้นที่)

คล้าย Line Plot แต่แรเงาพื้นที่ใต้เส้น เหมาะสำหรับแสดงปริมาณสะสม:

```
df.plot(x='Year', y='GDP_PPP', kind='area')  
df.plot(x='Year', y='GDP_Nominal', kind='area')
```

4.3.1.5 Bar Chart (แผนภูมิแท่ง)

ใช้สำหรับเปรียบเทียบข้อมูลเชิงหมวดหมู่ รองรับทั้งแนวตั้ง (bar) และแนวนอน (barh):

```
df.plot(x='Year', y='GDP_PerCapita_PPP',  
kind='bar')  
df.plot(x='Year', y='GDP_PerCapita_Nominal',  
kind='barh')
```

4.3.1.6 Histogram (ฮิสโตแกรม)

ใช้สำหรับแสดงการกระจายความถี่ของข้อมูล สามารถกำหนดจำนวน bins ได้:

```
df['Inflation'].plot(kind='hist')  
df['Inflation'].plot(kind='hist', bins=100)  
df['Unemployment'].plot(kind='hist')  
df['Unemployment'].plot(kind='kde') #  
แสดงเส้นโค้งความหนาแน่น
```

4.3.1.7 Box Plot (กราฟกล่อง)

ใช้สำหรับแสดงการกระจายของข้อมูลและตรวจจับค่าผิดปกติ (Outliers):

```
df['Inflation'].plot(kind='box')
df['Unemployment'].plot(kind='box')
```

4.3.1.8 Pie Chart (แผนภูมิวงกลม)

ใช้สำหรับแสดงสัดส่วนของข้อมูลเชิงหมวดหมู่:

```
df['Inflation_Change'].value_counts()
df['Inflation_Change'].value_counts().plot(kind='pie')
```

ผลลัพธ์ Inflation_Change: Decrease (23), Increase (21), No change (3)

4.3.1.9 Color Map

ใช้ Scatter Plot พร้อมเพื่มนมิติที่สามด้วยสี
ช่วยให้เห็นความสัมพันธ์ของสามตัวแปรพร้อมกัน:

```
df.plot.scatter(x='Year', y='Inflation',
               c='Unemployment')
```

4.3.1.10 เอกสารอ้างอิง

รายละเอียดเพิ่มเติมสามารถค้นหาได้ที่:

<https://pandas.pydata.org/pandas->

[docs/stable/reference/api/pandas.DataFrame.plot.html](https://pandas.pydata.org/pandas-)

https://pandas.pydata.org/docs/user_guide/visualization.html

4.4 การแสดงผลข้อมูลด้วย Matplotlib

แม้ Pandas จะมีการรองรับการแสดงผลข้อมูลพื้นฐาน
แต่ผู้ใช้อาจต้องการเครื่องมือที่ทรงพลังกว่า สำหรับการปรับแต่งกราฟ Matplotlib

เป็นไลบรารี Python อเนกประสงค์ที่ให้การควบคุมการแสดงผลได้อย่างสมบูรณ์ Matplotlib มีชื่อเสียงด้านตัวเลือกการปรับแต่งที่หลากหลาย ตั้งแต่การปรับสี Marker และรูปแบบเส้น ไปจนถึงการควบคุมสเกลแกนและหมายเหตุประกอบ

4.4.1 แบบฝึกหัด – การแสดงผลข้อมูลด้วย Matplotlib

เอกสารอ้างอิง: <https://matplotlib.org>

4.4.1.1 การตั้งค่า

```
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
df = pd.read_csv('/content/Economy_of_US.csv')
```

4.4.1.2 กราฟพื้นฐาน

สร้างกราฟง่ายๆ ด้วย plt.plot() พร้อมกำหนดชื่อแกนและหัวเรื่อง:

```
plt.plot(df['Year'], df['GDP_Growth'])
plt.xlabel('Year')
plt.ylabel('GDP Growth')
plt.title('Economy of US')
```

การเปลี่ยน Marker: Matplotlib รองรับ Marker หลากหลายรูปแบบ เช่น 'o', '*', '!', ',', 'x', 'X', '+', 'P', 's', 'D', 'd', 'p', 'H', 'h', 'V', '^', '<', '>' เป็นต้น:

```
plt.plot(df['Year'], df['GDP_Growth'], 'o') #
Marker วงกลม
for m in markers:
    plt.plot(df['Year'], df['GDP_Growth'], m)
plt.show()
```

การเปลี่ยนรูปแบบเส้น: รองรับรูปแบบ '-' (เส้นต่อเนื่อง), '.' (เส้นจุด), '--' (เส้นประ), '-.' (เส้นสลับ):

```
line = ['-', ':', '--', '-.']
for l in line:
    plt.plot(df['Year'], df['GDP_Growth'], 'o' + l
             + 'r')
```

การเปลี่ยนสี: รองรับสี 'r' (แดง), 'g' (เขียว), 'b' (น้ำเงิน), 'c' (ฟ้าอมเขียว), 'm' (ม่วงแดง), 'y' (เหลือง), 'k' (ดำ), 'w' (ขาว):

```
color = ['r', 'g', 'b', 'c', 'm', 'y', 'k', 'w']
for c in color:
    plt.plot(df['Year'], df['GDP_Growth'], 'o:' +
             c)
```

การเปลี่ยนขนาด Marker: ใช้พารามิเตอร์ ms (markersize):

```
plt.plot(df['Year'], df['GDP_Growth'],
         marker='^', ms=15)
```

4.4.1.3 Scatter Plot

ใช้ plt.scatter() สำหรับกราฟกระจาย รองรับการเพิ่ม Colorbar และการปรับขนาดจุดตามค่าข้อมูล:

```
plt.scatter(df['Year'], df['GDP_Growth'])

# เพิ่ม Colorbar
plt.scatter(df['Year'], df['GDP_Growth'],
            c=df['Inflation'], cmap='hot')
plt.colorbar()

# ปรับขนาดจุดตาม Unemployment
plt.scatter(df['Year'], df['GDP_Growth'],
            s=df['Unemployment']*1000)

# รวม Colorbar และขนาดจุด
plt.scatter(df['Year'], df['GDP_Growth'],
            c=df['Inflation'], cmap='hot', alpha=0.5,
            s=df['Unemployment']*1000)
```

4.4.1.4 Pie Plot

```
plt.pie(df['Inflation_Change'].value_counts(),
        labels=['Decrease', 'Increase', 'No change'])

plt.legend()
```

สามารถเพิ่ม explode เพื่อดึงชิ้นส่วนออกมาเน้น:

```
plt.pie(df['Inflation_Change'].value_counts(),
        labels=['Decrease', 'Increase', 'No change'],
        explode=[0.0, 0.2, 0])
```

4.4.1.5 Box Plot และ Violin Plot

```
plt.boxplot(df['Inflation'])
plt.violinplot(df['Inflation'])
```

4.4.1.6 Bar Plot และ Histogram

```
plt.bar(df['Year'], df['GDP_Growth'])
plt.hist(df['GDP_Growth'])
plt.hist(df['GDP_Growth'], bins=50)
```

4.4.1.7 Multi-Plots (กราฟหลายชุด)

สามารถวาดหลายกราฟในพื้นที่เดียวกัน หรือแบ่งเป็น Subplot:

```
plt.plot(df['Year'], df['GDP_Growth'],
        label='GDP_Growth')
plt.plot(df['Year'], df['Inflation'],
        label='Inflation')
plt.plot(df['Year'], df['Unemployment'],
        label='Unemployment')
plt.grid()
plt.legend()
```

การจัดเรียงแบบ Subplot แนวตั้ง (3 แถว 1 คอลัมน์):

```
plt.subplot(3, 1, 1) # แถวที่ 1
plt.subplot(3, 1, 2) # แถวที่ 2
```

```
plt.subplot(3, 1, 3) # แถวที่ 3
```

การจัดเรียงแบบ Subplot แนวนอน (1 แถว 3 คอลัมน์):

```
plt.subplot(1, 3, 1) # คอลัมน์ที่ 1
```

```
plt.subplot(1, 3, 2) # คอลัมน์ที่ 2
```

```
plt.subplot(1, 3, 3) # คอลัมน์ที่ 3
```

4.5 การแสดงผลข้อมูลด้วย Seaborn

Seaborn เป็นไลบรารีการแสดงผลข้อมูล Python ที่สร้างบน Matplotlib มีความสมดุลระหว่าง ความสะดวกใช้งานและสุนทรียศาสตร์เริ่มต้นที่สวยงาม Seaborn เป็นที่รู้จักในด้านการสร้างกราฟ สถิติที่น่าดึงดูดและให้ข้อมูลเชิงลึกโดยอัตโนมัติ รองรับ Color Palette และ Theme ที่หลากหลายสำหรับการปรับแต่ง

4.5.1 แบบฝึกหัด – การแสดงผลข้อมูลด้วย Seaborn

เอกสารอ้างอิง: <https://seaborn.pydata.org>

4.5.1.1 การตั้งค่า

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
tips = sns.load_dataset('tips')
```

ชุดข้อมูล tips ประกอบด้วยข้อมูลทิปในร้านอาหาร ได้แก่ total_bill, tip, sex, smoker, day, time และ size

4.5.1.2 Relational Plots (กราฟความสัมพันธ์)

ใช้ `sns.relplot()` สำหรับแสดงความสัมพันธ์ระหว่างสองตัวแปร รองรับการแบ่งกลุ่มด้วย hue และ col:

```
sns.relplot(data=tips, x='total_bill', y='tip')
sns.relplot(data=tips, x='total_bill', y='tip',
             hue='sex') # แบ่งตามเพศ

sns.relplot(data=tips, x='total_bill', y='tip',
             hue='day') # แบ่งตามวัน

sns.relplot(data=tips, x='total_bill', y='tip',
             hue='time') # แบ่งตามช่วงเวลา

sns.relplot(data=tips, x='total_bill', y='tip',
             hue='smoker', col='time') # หลายคอลัมน์
```

สามารถปรับขนาดจุดตามค่าข้อมูลและเพิ่มความโปร่งใสด้วย alpha:

```
sns.relplot(data=tips, x='total_bill', y='tip',
            size='size', sizes=(15,200), alpha=0.5,
            hue='size')
```

รองรับ Line Plot ด้วยการกำหนด kind='line':

```
sns.relplot(data=tips, x='day', y='total_bill',
            kind='line')

sns.relplot(data=tips, x='day', y='total_bill',
            hue='sex', kind='line', col='time')
```

4.5.1.3 Distribution Plots (กราฟการกระจาย)

ใช้ sns.displot() สำหรับแสดงการกระจายของข้อมูล รองรับหลายรูปแบบ:

```
sns.displot(data=tips, x='total_bill') # Histogram

sns.displot(data=tips, x='total_bill',
            col='sex') # แบ่งตามเพศ

sns.displot(data=tips, x='total_bill',
            hue='sex') # สีต่างกันตามเพศ

sns.displot(data=tips, x='total_bill',
            hue='sex', kind='kde') # KDE

sns.displot(data=tips, x='total_bill',
            hue='sex', kind='kde', multiple='stack') # แบบ Stack
```

```
sns.displot(data=tips, x='total_bill', y='tip',
kind='kde') # KDE สองตัวแปร

sns.displot(data=tips, x='total_bill', y='tip',
kind='kde', rug=True) # พร้อม Rug

sns.displot(data=tips, x='total_bill',
kind='ecdf') # ECDF

sns.displot(data=tips, x='total_bill',
kind='ecdf', hue='sex', col='day')
```

4.5.1.4 Categorical Plots (กราฟเชิงหมวดหมู่)

ใช้ `sns.catplot()` สำหรับแสดงข้อมูลเชิงหมวดหมู่ รองรับ kind หลายประเภท:

```
sns.catplot(data=tips, x='day', y='total_bill')
# Strip Plot (default)

sns.catplot(data=tips, x='day', y='total_bill',
hue='sex') # แบ่งตามเพศ

sns.catplot(data=tips, x='day', y='total_bill',
kind='box') # Box Plot

sns.catplot(data=tips, x='day', y='total_bill',
kind='box', hue='sex')

sns.catplot(data=tips, x='day', y='total_bill',
kind='violin') # Violin Plot

sns.catplot(data=tips, x='day', y='total_bill',
kind='violin', hue='sex', split=True)

sns.violinplot(data=tips, x='day',
y='total_bill', hue='sex', split=True,
inner='quartile')

sns.catplot(data=tips, x='day', y='total_bill',
kind='bar') # Bar Plot

sns.catplot(data=tips, x='day', y='total_bill',
kind='bar', hue='sex')
```

4.5.1.5 Joint Plots (กราฟรวม)

ใช้ `sns.jointplot()` สำหรับแสดงการกระจายของสองตัวแปรพร้อมกัน รองรับ kind หลายประเภท:

```
sns.jointplot(data=tips, x='total_bill',
              y='tip', hue='sex', kind='kde')
sns.jointplot(data=tips, x='total_bill',
              y='tip', hue='sex', kind='scatter')
```

รองรับ kind ทั้งหมด: scatter, hist, hex, kde, reg, resid:

```
kind = ['scatter', 'hist', 'hex', 'kde', 'reg',
        'resid']
for k in kind:
    sns.jointplot(data=tips, x='total_bill',
                  y='tip', kind=k)
```

4.5.1.6 Pair Plots (กราฟคู่)

ใช้ `sns.pairplot()`

สำหรับแสดงความสัมพันธ์ระหว่างตัวแปรตัวเลขทุกคู่ในชุดข้อมูลพร้อมกัน:

```
sns.pairplot(data=tips) # แสดงทุกคู่ตัวแปร
sns.pairplot(data=tips, hue='sex') # แบ่งสีตามเพศ
```